

Review Questions

The answers to the chapter review questions can be found in the Appendix.

1. Which code can be inserted to have the code print 2?

```
public class BirdSeed {
    private int numberBags;
    boolean call;

    public BirdSeed() {
        // LINE 1
        call = false;
        // LINE 2
    }

    public BirdSeed(int numberBags) {
        this.numberBags = numberBags;
    }

    public static void main(String[] args) {
        var seed = new BirdSeed();
        System.out.print(seed.numberBags);
    } }
```

- A. Replace line 1 with `BirdSeed(2);`.
 - B. Replace line 2 with `BirdSeed(2);`.
 - C. Replace line 1 with `new BirdSeed(2);`.
 - D. Replace line 2 with `new BirdSeed(2);`.
 - E. Replace line 1 with `this(2);`.
 - F. Replace line 2 with `this(2);`.
 - G. The code prints 2 without any changes.
2. Which modifier pairs can be used together in a method declaration? (Choose all that apply.)
- A. `static` and `final`
 - B. `private` and `static`
 - C. `static` and `abstract`
 - D. `private` and `abstract`
 - E. `abstract` and `final`
 - F. `private` and `final`

3. Which of the following statements about methods are true? (Choose all that apply.)

- A. Overloaded methods must have the same signature.
- B. Overridden methods must have the same signature.
- C. Hidden methods must have the same signature.
- D. Overloaded methods must have the same return type.
- E. Overridden methods must have the same return type.
- F. Hidden methods must have the same return type.

4. What is the output of the following program?

```
1: class Mammal {  
2:     private void sneeze() {}  
3:     public Mammal(int age) {  
4:         System.out.print("Mammal");  
5:     } }  
6: public class Platypus extends Mammal {  
7:     int sneeze() { return 1; }  
8:     public Platypus() {  
9:         System.out.print("Platypus");  
10:    }  
11:    public static void main(String[] args) {  
12:        new Mammal(5);  
13:    } }
```

- A. Platypus
- B. Mammal
- C. PlatypusMammal
- D. MammalPlatypus
- E. The code will compile if line 7 is changed.
- F. The code will compile if line 9 is changed.

5. Which of the following complete the constructor so that this code prints out 50? (Choose all that apply.)

```
class Speedster {  
    int numSpots;  
}  
public class Cheetah extends Speedster {  
    int numSpots;  
  
    public Cheetah(int numSpots) {  
        // INSERT CODE HERE  
    }  
}
```

```
public static void main(String[] args) {  
    Speedster s = new Cheetah(50);  
    System.out.print(s.numSpots);  
}  
}
```

- A. numSpots = numSpots;
 - B. numSpots = this.numSpots;
 - C. this.numSpots = numSpots;
 - D. numSpots = super.numSpots;
 - E. super.numSpots = numSpots;
 - F. The code does not compile regardless of the code inserted into the constructor.
 - G. None of the above
6. Which of the following declare immutable classes? (Choose all that apply.)

```
public final class Moose {  
    private final int antlers;  
}
```

```
public class Caribou {  
    private int antlers = 10;  
}
```

```
public class Reindeer {  
    private final int antlers = 5;  
}
```

```
public final class Elk {}
```

```
public final class Deer {  
    private final Object o = new Object();  
}
```

- A. Moose
- B. Caribou
- C. Reindeer
- D. Elk
- E. Deer
- F. None of the above

7. What is the output of the following code?

```
1: class Arthropod {
2:     protected void printName(long input) {
3:         System.out.print("Arthropod");
4:     }
5:     void printName(int input) {
6:         System.out.print("Spooky");
7:     } }
8: public class Spider extends Arthropod {
9:     protected void printName(int input) {
10:        System.out.print("Spider");
11:    }
12:    public static void main(String[] args) {
13:        Arthropod a = new Spider();
14:        a.printName((short)4);
15:        a.printName(4);
16:        a.printName(5L);
17:    } }
```

- A. SpiderSpiderArthropod
- B. SpiderSpiderSpider
- C. SpiderSpookyArthropod
- D. SpookySpiderArthropod
- E. The code will not compile because of line 5.
- F. The code will not compile because of line 9.
- G. None of the above

8. What is the result of the following code?

```
1: abstract class Bird {
2:     private final void fly() { System.out.println("Bird"); }
3:     protected Bird() { System.out.print("Wow-"); }
4: }
5: public class Pelican extends Bird {
6:     public Pelican() { System.out.print("Oh-"); }
7:     protected void fly() { System.out.println("Pelican"); }
8:     public static void main(String[] args) {
9:         var chirp = new Pelican();
10:        chirp.fly();
11:    } }
```

- A. Oh-Bird
 - B. Oh-Pelican
 - C. Wow-Oh-Bird
 - D. Wow-Oh-Pelican**
 - E. The code contains a compilation error.
 - F. None of the above
9. Which of the following statements about overridden methods are true? (Choose all that apply.)
- A. An overridden method must contain method parameters that are the same or covariant with the method parameters in the inherited method.
 - B. An overridden method may declare a new exception, provided it is not checked.**
 - C. An overridden method must be more accessible than the method in the parent class.
 - D. An overridden method may declare a broader checked exception than the method in the parent class.
 - E. If an inherited method returns void, then the overridden version of the method must return void.**
 - F. None of the above
10. Which of the following pairs, when inserted into the blanks, allow the code to compile? (Choose all that apply.)
- ```

1: public class Howler {
2: public Howler(long shadow) {
3: _____;
4: }
5: private Howler(int moon) {
6: super();
7: }
8: }
9: class Wolf extends Howler {
10: protected Wolf(String stars) {
11: super(2L);
12: }
13: public Wolf() {
14: _____;
15: }
16: }
```
- A. this(3) at line 3, this("") at line 14**
  - B. this() at line 3, super(1) at line 14
  - C. this((short)1) at line 3, this(null) at line 14**
  - D. super() at line 3, super() at line 14

- E. `this(2L)` at line 3, `super((short)2)` at line 14
- F. `this(5)` at line 3, `super(null)` at line 14
- G. Remove lines 3 and 14.

11. What is the result of the following?

```
1: public class PolarBear {
2: StringBuilder value = new StringBuilder("t");
3: { value.append("a"); }
4: { value.append("c"); }
5: private PolarBear() {
6: value.append("b");
7: }
8: public PolarBear(String s) {
9: this();
10: value.append(s);
11: }
12: public PolarBear(CharSequence p) {
13: value.append(p);
14: }
15: public static void main(String[] args) {
16: Object bear = new PolarBear();
17: bear = new PolarBear("f");
18: System.out.println(((PolarBear)bear).value);
19: } }
```

- A. `tacb`
- B. `tacf`
- C. `tacbf`
- D. `tcafb`
- E. `taftacb`
- F. The code does not compile.
- G. An exception is thrown.

12. How many lines of the following program contain a compilation error?

```
1: public class Rodent {
2: public Rodent(Integer x) {}
3: protected static Integer chew() throws Exception {
4: System.out.println("Rodent is chewing");
5: return 1;
6: }
```

```
7: }
8: class Beaver extends Rodent {
9: public Number chew() throws RuntimeException {
10: System.out.println("Beaver is chewing on wood");
11: return 2;
12: } }
```

**A.** None

**B.** 1

**C.** 2

**D.** 3

**E.** 4

**F.** 5

- 13.** Which of these classes compile and will include a default constructor created by the compiler? (Choose all that apply.)

**A.**

```
public class Bird {}
```

**B.**

```
public class Bird {
 public bird() {}
}
```

**C.**

```
public class Bird {
 public bird(String name) {}
}
```

**D.**

```
public class Bird {
 public Bird() {}
}
```

**E.**

```
public class Bird {
 Bird(String name) {}
}
```

**F.**

```
public class Bird {
 private Bird(int age) {}
}
```

**G.**

```
public class Bird {
 public Bird bird() { return null; }
}
```

14. Which of the following statements about inheritance are correct? (Choose all that apply.)

- A. A class can directly extend any number of classes.
- B. A class can implement any number of interfaces.
- C. All variables inherit `java.lang.Object`.
- D. If class A is extended by B, then B is a superclass of A.
- E. If class C implements interface D, then C is a subtype of D.
- F. Multiple inheritance is the property of a class to have multiple direct superclasses.

15. Which statements about the following program are correct? (Choose all that apply.)

```
1: abstract class Nocturnal {
2: boolean isBlind();
3: }
4: public class Owl extends Nocturnal {
5: public boolean isBlind() { return false; }
6: public static void main(String[] args) {
7: var nocturnal = (Nocturnal)new Owl();
8: System.out.println(nocturnal.isBlind());
9: } }
```

- A. It compiles and prints true.
- B. It compiles and prints false.
- C. The code will not compile because of line 2.
- D. The code will not compile because of line 5.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.
- G. None of the above

16. What is the result of the following?

```
1: class Arachnid {
2: static StringBuilder sb = new StringBuilder();
3: { sb.append("c"); }
4: static
5: { sb.append("u"); }
6: { sb.append("r"); }
7: }
```



```

8: public class Scorpion extends Arachnid {
9: static
10: { sb.append("q"); }
11: { sb.append("m"); }
12: public static void main(String[] args) {
13: System.out.print(Scorpion.sb + " ");
14: System.out.print(Scorpion.sb + " ");
15: new Arachnid();
16: new Scorpion();
17: System.out.print(Scorpion.sb);
18: } }

```

- A. qu qu qumrcrc
  - B. u u ucrcrm
  - C. uq uq uqmcrcr
  - D. uq uq uqrcrm**
  - E. qu qu qumrcrc
  - F. qu qu qucrarm
  - G. The code does not compile.
17. Which of the following are true? (Choose all that apply.)
- A. `this()` can be called from anywhere in a constructor.
  - B. `this()` can be called from anywhere in an instance method.
  - C. `this.variableName` can be called from any instance method in the class.**
  - D. `this.variableName` can be called from any static method in the class.
  - E. You can call the default constructor written by the compiler using `this()`.
  - F. You can access a private constructor with the `main()` method in the same class.**
18. Which statements about the following classes are correct? (Choose all that apply.)
- ```

1: public class Mammal {
2:     private void eat() {}
3:     protected static void drink() {}
4:     public Integer dance(String p) { return null; }
5: }
6: class Primate extends Mammal {
7:     public void eat(String p) {}
8: }
9: class Monkey extends Primate {

```

```
10: public static void drink() throws RuntimeException {}
11: public Number dance(CharSequence p) { return null; }
12: public int eat(String p) {}
13: }
```

- A. The eat() method in Mammal is correctly overridden on line 7.
- B. The eat() method in Mammal is correctly overloaded on line 7.
- C. The drink() method in Mammal is correctly overridden on line 10.
- D. The drink() method in Mammal is correctly hidden on line 10.
- E. The dance() method in Mammal is correctly overridden on line 11.
- F. The dance() method in Mammal is correctly overloaded on line 11.
- G. The eat() method in Primate is correctly hidden on line 12.
- H. The eat() method in Primate is correctly overloaded on line 12.

19. What is the output of the following code?

```
1: class Reptile {
2:     {System.out.print("A");}
3:     public Reptile(int hatch) {}
4:     void layEggs() {
5:         System.out.print("Reptile");
6:     } }
7: public class Lizard extends Reptile {
8:     static {System.out.print("B");}
9:     public Lizard(int hatch) {}
10:    public final void layEggs() {
11:        System.out.print("Lizard");
12:    }
13:    public static void main(String[] args) {
14:        var reptile = new Lizard(1);
15:        reptile.layEggs();
16:    } }
```

- A. AALizard
- B. BALizard
- C. BLizardA
- D. ALizard
- E. The code will not compile because of line 3.
- F. None of the above

20. Which statement about the following program is correct?

```
1: class Bird {  
2:     int feathers = 0;  
3:     Bird(int x) { this.feathers = x; }  
4:     Bird fly() {  
5:         return new Bird(1);  
6:     } }  
7: class Parrot extends Bird {  
8:     protected Parrot(int y) { super(y); }  
9:     protected Parrot fly() {  
10:         return new Parrot(2);  
11:     } }  
12: public class Macaw extends Parrot {  
13:     public Macaw(int z) { super(z); }  
14:     public Macaw fly() {  
15:         return new Macaw(3);  
16:     }  
17:     public static void main(String... sing) {  
18:         Bird p = new Macaw(4);  
19:         System.out.print(((Parrot)p.fly()).feathers);  
20:     } }
```

- A. One line contains a compiler error.
 - B. Two lines contain compiler errors.
 - C. Three lines contain compiler errors.
 - D. The code compiles but throws a `ClassCastException` at runtime.
 - E. The program compiles and prints 3.
 - F. The program compiles and prints 0.
21. Which of the following are properties of immutable classes? (Choose all that apply.)
- A. The class can contain setter methods, provided they are marked `final`.
 - B. The class must not be able to be extended outside the class declaration.
 - C. The class may not contain any instance variables.
 - D. The class must be marked `static`.
 - E. The class may not contain any `static` variables.
 - F. The class may only contain `private` constructors.
 - G. The data for mutable instance variables may be read, provided they cannot be modified by the caller.

22. What does the following program print?

```
1: class Person {
2:     static String name;
3:     void setName(String q) { name = q; } }
4: public class Child extends Person {
5:     static String name;
6:     void setName(String w) { name = w; }
7:     public static void main(String[] p) {
8:         final Child m = new Child();
9:         final Person t = m;
10:        m.name = "Elysia";
11:        t.name = "Sophia";
12:        m.setName("Webby");
13:        t.setName("Olivia");
14:        System.out.println(m.name + " " + t.name);
15:    } }
```

- A. Elysia Sophia
- B. Webby Olivia
- C. Olivia Olivia
- D. Olivia Sophia
- E. The code does not compile.
- F. None of the above

23. What is the output of the following program?

```
1: class Canine {
2:     public Canine(boolean t) { logger.append("a"); }
3:     public Canine() { logger.append("q"); }
4:
5:     private StringBuilder logger = new StringBuilder();
6:     protected void print(String v) { logger.append(v); }
7:     protected String view() { return logger.toString(); }
8: }
9:
10: class Fox extends Canine {
11:     public Fox(long x) { print("p"); }
12:     public Fox(String name) {
13:         this(2);
14:         print("z");
15:     }
```

```
16: }
17:
18: public class Fennec extends Fox {
19:     public Fennec(int e) {
20:         super("tails");
21:         print("j");
22:     }
23:     public Fennec(short f) {
24:         super("eevee");
25:         print("m");
26:     }
27:
28:     public static void main(String... unused) {
29:         System.out.println(new Fennec(1).view());
30:     } }
```

- A. qpz
- B. qpzj**
- C. jzpa
- D. apj
- E. apjm
- F. The code does not compile.
- G. None of the above

24. What is printed by the following program?

```
1: class Antelope {
2:     public Antelope(int p) {
3:         System.out.print("4");
4:     }
5:     { System.out.print("2"); }
6:     static { System.out.print("1"); }
7: }
8: public class Gazelle extends Antelope {
9:     public Gazelle(int p) {
10:         super(6);
11:         System.out.print("3");
12:     }
```

```
13: public static void main(String hopping[]) {  
14:     new Gazelle(0);  
15: }  
16: static { System.out.print("8"); }  
17: { System.out.print("9"); }  
18: }
```

- A. 182640
- B. 182943
- C. 182493
- D. 421389
- E. The code does not compile.
- F. The output cannot be determined until runtime.

25. Which of the following are true about a concrete class? (Choose all that apply.)

- A. A concrete class can be declared as `abstract`.
- B. A concrete class must implement all inherited abstract methods.
- C. A concrete class can be marked as `final`.
- D. A concrete class must be immutable.
- E. A concrete method that implements an abstract method must match the method declaration of the abstract method exactly.

26. What is the output of the following code?

```
4: public abstract class Whale {  
5:     public abstract void dive();  
6:     public static void main(String[] args) {  
7:         Whale whale = new Orca();  
8:         whale.dive(3);  
9:     }  
10: }  
11: class Orca extends Whale {  
12:     static public int MAX = 3;  
13:     public void dive() {  
14:         System.out.println("Orca diving");  
15:     }  
16:     public void dive(int... depth) {  
17:         System.out.println("Orca diving deeper " + MAX);  
18:     } }
```

- A. Orca diving
- B. Orca diving deeper 3
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 8.
- E. The code will not compile because of line 11.
- F. The code will not compile because of line 12.
- G. The code will not compile because of line 17.
- H. None of the above

Respuestas del cuestionario semana 4 capítulo 7	
Pregunta	Comentario
1	Gecko compila porque los records son implícitamente final y no pueden ser abstract. BeardedDragon compila porque un record puede sobrescribir métodos accessor.
2	TurtleFrog puede asignarse a cualquier supertipo suyo: Frog, TurtleFrog, CanHop o var.
3	Falta el punto y coma después de los valores del enum cuando existen otros miembros.
4	Una clase que extiende una sealed class debe declararse final, sealed o non-sealed.
5	Beetle no implementa correctamente todos los métodos abstractos heredados.
6	Un método abstracto no puede tener cuerpo y una clase implementa una interfaz, no la extiende.
7	El método implementado debe ser public para coincidir con la interfaz.
8	Las variables de instancia deben ser private para una encapsulación adecuada.
9	Se aceptan instancias de Cobra, GardenSnake o null.
10	En X cualquier tipo válido funciona; en Z debe ser ArrayList o subtipo.
11	La salida es 10 porque se usa la variable butter de la clase interna.
12	La encapsulación permite getters y setters y requiere variables privadas.
13	Los case de un enum no deben incluir el nombre del enum.
14	Las sealed interfaces restringen extensiones e implementaciones; las sealed classes pueden extenderse por clases abstractas.
15	El código no compila porque Spirit es final y no puede extenderse mediante una clase anónima.
16	Una clase nested static no puede acceder a miembros de instancia de la clase externa.
17	Los métodos de interfaz con cuerpo deben ser default o private; protected no es válido.
18	No puede instanciarse la clase interna Diet desde un contexto estático sin una instancia de Deer.
19	Todos los valores del enum deben implementar el método abstracto.

20	Los métodos sobrescritos se resuelven en tiempo de ejecución; final evita override; los métodos ocultos dependen del tipo de referencia.
21	Un constructor no compacto de record debe invocar otro constructor como primera instrucción.
22	Son las sintaxis válidas para crear instancias de clases internas y nested static.
23	La sintaxis correcta es Swim.super.perform().
24	Los métodos static no pueden llamar métodos de instancia directamente.
25	Imprime: x is 24.
26	Los constructores de enum son implícitamente private y falta un punto y coma después de los valores.
27	Los records generan accessors, equals(), hashCode() y toString().
28	Camel, EatsGrass y Eagle contienen errores de compilación.
29	Existen tres errores de compilación.
30	EmperorTamarin y Friendly contienen errores de compilación.

Review Questions

The answers to the chapter review questions can be found in the Appendix.

1. Which of the following are valid record declarations? (Choose all that apply.)

```
public record Iguana(int age) {  
    private static final int age = 10; }  
  
public final record Gecko() {}  
  
public abstract record Chameleon() {  
    private static String name; }  
  
public record BeardedDragon(boolean fun) {  
    @Override public boolean fun() { return false; } }  
  
public record Newt(long size) {  
    @Override public boolean equals(Object obj) { return false; }  
    public void setSize(long size) {  
        this.size = size;  
    } } }
```

- A. Iguana
B. Gecko
C. Chameleon
D. BeardedDragon
E. Newt
F. None of the above
2. Which of the following statements can be inserted in the blank line so that the code will compile successfully? (Choose all that apply.)

```
interface CanHop {}  
public class Frog implements CanHop {  
    public static void main(String[] args) {  
        _____ frog = new TurtleFrog();  
    }  
}  
class BrazilianHornedFrog extends Frog {}  
class TurtleFrog extends Frog {}
```

- A. Frog
- B. TurtleFrog
- C. BrazilianHornedFrog
- D. CanHop
- E. var
- F. Long
- G. None of the above; the code contains a compilation error.

3. What is the result of the following program?

```
public class Favorites {  
    enum Flavors {  
        VANILLA, CHOCOLATE, STRAWBERRY  
        static final Flavors DEFAULT = STRAWBERRY;  
    }  
    public static void main(String[] args) {  
        for(final var e : Flavors.values())  
            System.out.print(e.ordinal()+" ");  
    }  
}
```

- A. 0 1 2
- B. 1 2 3
- C. Exactly one line of code does not compile.
- D. More than one line of code does not compile.
- E. The code compiles but produces an exception at runtime.
- F. None of the above

4. What is the output of the following program?

```
public sealed class ArmoredAnimal permits Armadillo {  
    public ArmoredAnimal(int size) {}  
    @Override public String toString() { return "Strong"; }  
    public static void main(String[] a) {  
        var c = new Armadillo(10, null);  
        System.out.println(c);  
    }  
}  
  
class Armadillo extends ArmoredAnimal {  
    @Override public String toString() { return "Cute"; }  
    public Armadillo(int size, String name) {  
        super(size);  
    }  
}
```

- A. Strong
 - B. Cute
 - C. The program does not compile.
 - D. The code compiles but produces an exception at runtime.
 - E. None of the above
5. Which statements about the following program are correct? (Choose all that apply.)
- ```
1: interface HasExoskeleton {
2: double size = 2.0f;
3: abstract int getNumberOfSections();
4: }
5: abstract class Insect implements HasExoskeleton {
6: abstract int getNumberOfLegs();
7: }
8: public class Beetle extends Insect {
9: int getNumberOfLegs() { return 6; }
10: int getNumberOfSections(int count) { return 1; }
11: }
```
- A. It compiles without issue.
  - B. The code will produce a `ClassCastException` if called at runtime.
  - C. The code will not compile because of line 2.
  - D. The code will not compile because of line 5.
  - E. The code will not compile because of line 8.
  - F. The code will not compile because of line 10.
6. Which statements about the following program are correct? (Choose all that apply.)
- ```
1: public abstract interface Herbivore {  
2:     int amount = 10;  
3:     public void eatGrass();  
4:     public abstract int chew() { return 13; }  
5: }  
6:  
7: abstract class IsAPlant extends Herbivore {  
8:     Object eatGrass(int season) { return null; }  
9: }
```
- A. It compiles and runs without issue.
 - B. The code will not compile because of line 1.
 - C. The code will not compile because of line 2.

D. The code will not compile because of line 4.

E. The code will not compile because of line 7.

F. The code will not compile because line 8 contains an invalid method override.

7. What is the output of the following program?

```
1: interface Aquatic {  
2:     int getNumOfGills(int p);  
3: }  
4: public class ClownFish implements Aquatic {  
5:     String getNumOfGills() { return "14"; }  
6:     int getNumOfGills(int input) { return 15; }  
7:     public static void main(String[] args) {  
8:         System.out.println(new ClownFish().getNumOfGills(-1));  
9:     } }
```

A. 14

B. 15

C. The code will not compile because of line 4.

D. The code will not compile because of line 5.

E. The code will not compile because of line 6.

F. None of the above

8. When inserted in order, which modifiers can fill in the blank to create a properly encapsulated class? (Choose all that apply.)

```
public class Rabbits {  
    _____ int numRabbits = 0;  
    _____ void multiply() {  
        numRabbits *= 6;  
    }  
    _____ int getNumberOfRabbits() {  
        return numRabbits;  
    }  
}
```

A. private, public, and public

B. private, protected, and private

C. private, private, and protected

D. public, public, and public

E. The class cannot be properly encapsulated since multiply() does not begin with set.

F. None of the above

9. Which of the following statements can be inserted in the blank so that the code will compile successfully? (Choose all that apply.)

```
abstract class Snake {}
class Cobra extends Snake {}
class GardenSnake extends Cobra {}
public class SnakeHandler {
    private Snake snakey;
    public void setSnake(Snake mySnake) { this.snakey = mySnake; }
    public static void main(String[] args) {
        new SnakeHandler().setSnake(_____);
    }
}
```

- A. `new Cobra()`
 - B. `new Snake()`
 - C. `new Object()`
 - D. `new String("Snake")`
 - E. `new GardenSnake()`
 - F. `null`
 - G. None of the above. The class does not compile, regardless of the value inserted in the blank.
10. What types can be inserted in the blanks on the lines marked X and Z that allow the code to compile? (Choose all that apply.)

```
interface Walk { private static List move() { return null; } }
interface Run extends Walk { public ArrayList move(); }
class Leopard implements Walk {
    public _____ move() { // X
        return null;
    }
}
class Panther implements Run {
    public _____ move() { // Z
        return null;
    }
}
```

- A. `Integer` on the line marked X
- B. `ArrayList` on the line marked X
- C. `List` on the line marked X
- D. `List` on the line marked Z

E. ArrayList on the line marked Z

F. None of the above, since the Run interface does not compile

G. The code does not compile for a different reason.

11. What is the result of the following code? (Choose all that apply.)

```
1: public class Movie {  
2:     private int butter = 5;  
3:     private Movie() {}  
4:     protected class Popcorn {  
5:         private Popcorn() {}  
6:         public static int butter = 10;  
7:         public void startMovie() {  
8:             System.out.println(butter);  
9:         }  
10:    }  
11:    public static void main(String[] args) {  
12:        var movie = new Movie();  
13:        Movie.Popcorn in = new Movie().new Popcorn();  
14:        in.startMovie();  
15:    } }
```

A. The output is 5.

B. The output is 10.

C. Line 6 generates a compiler error.

D. Line 12 generates a compiler error.

E. Line 13 generates a compiler error.

F. The code compiles but produces an exception at runtime.

12. Which of the following are true about encapsulation? (Choose all that apply.)

A. It allows getters.

B. It allows setters.

C. It requires specific naming conventions.

D. It requires public instance variables.

E. It requires private instance variables.

13. What is the result of the following program?

```
public class Weather {  
    enum Seasons {  
        WINTER, SPRING, SUMMER, FALL  
    }  
}
```

```

public static void main(String[] args) {
    Seasons v = null;
    switch (v) {
        case Seasons.SPRING -> System.out.print("s");
        case Seasons.WINTER -> System.out.print("w");
        case Seasons.SUMMER -> System.out.print("m");
        default -> System.out.println("missing data"); }
    }
}

```

- A. s
 - B. w
 - C. m
 - D. missing data
 - E. Exactly one line of code does not compile.
 - F. More than one line of code does not compile.**
 - G. The code compiles but produces an exception at runtime.
14. Which statements about sealed classes are correct? (Choose all that apply.)
- A. A sealed interface restricts which subinterfaces may extend it.**
 - B. A sealed class cannot be indirectly extended by a class that is not listed in its `permits` clause.
 - C. A sealed class can be extended by an abstract class.**
 - D. A sealed class can be extended by a subclass that uses the `non-sealed` modifier.
 - E. A sealed interface restricts which subclasses may implement it.**
 - F. A sealed class cannot contain any nested subclasses.
 - G. None of the above
15. Which lines, when entered independently into the blank, allow the code to print `Not scared` at runtime? (Choose all that apply.)

```

public class Ghost {
    public static void boo() {
        System.out.println("Not scared");
    }
    protected final class Spirit {
        public void boo() {
            System.out.println("Booo!!!");
        }
    }
}

```



```

    public static void main(String... haunt) {
        var g = new Ghost().new Spirit() {};
        _____;
    }
}

```

- A. `g.boo()`
- B. `g.super.boo()`
- C. `new Ghost().boo()`
- D. `g.Ghost.boo()`
- E. `new Spirit().boo()`
- F. `Ghost.boo()`
- G. None of the above**

16. The following code appears in a file named `Ostrich.java`. What is the result of compiling the source file?

```

1: public class Ostrich {
2:     private int count;
3:     static class OstrichWrangler {
4:         public int stampede() {
5:             return count;
6:         } } }

```

- A. The code compiles successfully, and one bytecode file is generated: `Ostrich.class`.
- B. The code compiles successfully, and two bytecode files are generated: `Ostrich.class` and `OstrichWrangler.class`.
- C. The code compiles successfully, and two bytecode files are generated: `Ostrich.class` and `Ostrich$OstrichWrangler.class`.
- D. A compiler error occurs on line 3.
- E. A compiler error occurs on line 5.**

17. Which lines of the following interface declarations do not compile? (Choose all that apply.)

```

1: public interface Omnivore {
2:     int amount = 10;
3:     static boolean gather = true;
4:     static void eatGrass() {}
5:     int findMore() { return 2; }
6:     default float rest() { return 2; }
7:     protected int chew() { return 13; }
8:     private static void eatLeaves() {}
9: }

```

- A. All of the lines compile without issue.
- B. Line 2
- C. Line 3
- D. Line 4
- E. Line 5
- F. Line 6
- G. Line 7
- H. Line 8

18. What is printed by the following program?

```
public class Deer {
    enum Food {APPLES, BERRIES, GRASS}
    protected class Diet {
        private Food getFavorite() {
            return Food.BERRIES;
        }
    }
    public static void main(String[] seasons) {
        System.out.print(switch(new Diet().getFavorite()) {
            case APPLES -> "a";
            case BERRIES -> "b";
            default -> "c";
        });
    }
}
```

- A. a
- B. b
- C. c
- D. The code declaration of the Diet class does not compile.
- E. The main() method does not compile.
- F. The code compiles but produces an exception at runtime.
- G. None of the above

19. Which of the following are printed by the Bear program? (Choose all that apply.)

```
public class Bear {
    enum FOOD {
        BERRIES, INSECTS {
            public boolean isHealthy() { return true; }},
        FISH, ROOTS, COOKIES, HONEY;
        public abstract boolean isHealthy();
    }
}
```

```
public static void main(String[] args) {  
    System.out.print(FOOD.INSECTS);  
    System.out.print(FOOD.INSECTS.ordinal());  
    System.out.print(FOOD.INSECTS.isHealthy());  
    System.out.print(FOOD.COOKIE.isHealthy());  
}  
}
```

- A. insects
 - B. INSECTS
 - C. 0
 - D. 1
 - E. false
 - F. true
 - G. The code does not compile.
20. Which statements about polymorphism and method inheritance are correct? (Choose all that apply.)
- A. Given an arbitrary instance of a class, it cannot be determined until runtime which overridden method will be executed in a parent class.
 - B. It cannot be determined until runtime which hidden method will be executed in a parent class.
 - C. Marking a method `static` prevents it from being overridden or hidden.
 - D. Marking a method `final` prevents it from being overridden or hidden.
 - E. The reference type of the variable determines which overridden method will be called at runtime.
 - F. The reference type of the variable determines which hidden method will be called at runtime.
21. Given the following record declaration, which lines of code can fill in the blank and allow the code to compile? (Choose all that apply.)
- ```
public record RabbitFood(int size, String brand, LocalDate expires) {
 public static int MAX_STORAGE = 100;
 public RabbitFood() {
 _____;
 }
}
```
- A. `size = MAX_STORAGE`
  - B. `this.size = 10`

- C. `if(expires.isAfter(LocalDate.now())) throw new RuntimeException()`
- D. `if(brand==null) super.brand = "Unknown"`
- E. `throw new RuntimeException()`
- F. **None of the above**

22. Which of the following can be inserted in the `rest()` method? (Choose all that apply.)

```
public class Lion {
 class Cub {}
 static class Den {}
 static void rest() {
 _____;
 } }

```

- A. `Cub a = Lion.new Cub()`
- B. `Lion.Cub b = new Lion().Cub()`
- C. **`Lion.Cub c = new Lion().new Cub()`**
- D. **`var d = new Den()`**
- E. `var e = Lion.new Cub()`
- F. `Lion.Den f = Lion.new Den()`
- G. **`Lion.Den g = new Lion.Den()`**
- H. `var h = new Cub()`

23. Given the following program, what can be inserted into the blank line that would allow it to print `Swim!` at runtime?

```
interface Swim {
 default void perform() { System.out.print("Swim!"); }
}
interface Dance {
 default void perform() { System.out.print("Dance!"); }
}
public class Penguin implements Swim, Dance {
 public void perform() { System.out.print("Smile!"); }
 private void doShow() {
 _____;
 }
 public static void main(String[] eggs) {
 new Penguin().doShow();
 }
}

```

- A. `super.perform()`
- B. `Swim.perform()`
- C. `super.Swim.perform()`
- D. `Swim.super.perform()`
- E. The code does not compile regardless of what is inserted into the blank.
- F. The code compiles, but due to polymorphism, it is not possible to produce the requested output without creating a new object.

24. Which lines of the following interface do not compile? (Choose all that apply.)

```
1: public interface BigCat {
2: abstract String getName();
3: static int hunt() { getName(); return 5; }
4: default void climb() { rest(); }
5: private void roar() { getName(); climb(); hunt(); }
6: private static boolean sneak() { roar(); return true; }
7: private int rest() { return 2; };
8: }
```

- A. Line 2
- B. Line 3
- C. Line 4
- D. Line 5
- E. Line 6
- F. Line 7
- G. None of the above

25. What does the following program print?

```
1: public class Zebra {
2: private int x = 24;
3: public int hunt() {
4: String message = "x is ";
5: abstract class Stripes {
6: private int x = 0;
7: public void print() {
8: System.out.print(message + Zebra.this.x);
9: }
10: }
11: var s = new Stripes() {};
12: s.print();
13: return x;
}
```

```
14: }
15: public static void main(String[] args) {
16: new Zebra().hunt();
17: } }
```

- A. x is 0
- B. x is 24
- C. Line 6 generates a compiler error.
- D. Line 8 generates a compiler error.
- E. Line 11 generates a compiler error.
- F. None of the above

26. Which statements about the following enum are true? (Choose all that apply.)

```
1: public enum Animals {
2: MAMMAL(true), INVERTEBRATE(Boolean.FALSE), BIRD(false),
3: REPTILE(false), AMPHIBIAN(false), FISH(false) {
4: public int swim() { return 4; }
5: }
6: final boolean hasHair;
7: public Animals(boolean hasHair) {
8: this.hasHair = hasHair;
9: }
10: public boolean hasHair() { return hasHair; }
11: public int swim() { return 0; }
12: }
```

- A. Compiler error on line 2
- B. Compiler error on line 3
- C. Compiler error on line 7
- D. Compiler error on line 8
- E. Compiler error on line 10
- F. Compiler error on another line
- G. The code compiles successfully.

27. Assuming a record is defined with at least one field, which components does the compiler always insert, each of which may be overridden or redeclared? (Choose all that apply.)

- A. A no-argument constructor
- B. An accessor method for each field
- C. The `toString()` method
- D. The `equals()` method

- E. A mutator method for each field
- F. A sort method for each field
- G. The hashCode() method

28. Which of the following classes and interfaces do not compile? (Choose all that apply.)

```
public abstract class Camel { void travel(); }

public interface EatsGrass { private abstract int chew(); }

public abstract class Elephant {
 abstract private class SleepsAlot {
 abstract int sleep();
 } }

public class Eagle { abstract soar(); }

public interface Spider { default void crawl() {} }
```

- A. Camel
- B. EatsGrass
- C. Elephant
- D. Eagle
- E. Spider
- F. None of the classes or interfaces compile.

29. How many lines of the following program contain a compilation error?

```
1: class Primate {
2: protected int age = 2;
3: { age = 1; }
4: public Primate() {
5: this().age = 3;
6: }
7: }
8: public class Orangutan {
9: protected int age = 4;
10: { age = 5; }
11: public Orangutan() {
12: this().age = 6;
13: }
14: public static void main(String[] bananas) {
```

```
15: final Primate x = (Primate)new Orangutan();
16: System.out.println(x.age);
17: }
18: }
```

- A. None, and the program prints 1 at runtime.
- B. None, and the program prints 3 at runtime.
- C. None, but it causes a `ClassCastException` at runtime.
- D. 1
- E. 2
- F. 3**
- G. 4

30. Assuming the following classes are declared as top-level types in the same file, which classes contain compiler errors? (Choose all that apply.)

```
sealed class Bird {
 public final class Flamingo extends Bird {}
}
```

```
sealed class Monkey {}
```

```
class EmperorTamarin extends Monkey {}
```

```
non-sealed class Mandrill extends Monkey {}
```

```
sealed class Friendly extends Mandrill permits Silly {}
```

```
final class Silly {}
```

- A. Bird
- B. Monkey
- C. EmperorTamarin**
- D. Mandrill
- E. Friendly**
- F. Silly
- G. All of the classes compile without issue.



| Respuestas del cuestionario semana 4 capítulo 7 |                                                                                                                                                                 |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pregunta                                        | Comentario                                                                                                                                                      |
| 1                                               | Gecko compila porque los records son implícitamente final y no pueden ser abstract. BeardedDragon compila porque un record puede sobrescribir métodos accessor. |
| 2                                               | TurtleFrog puede asignarse a cualquier supertipo suyo: Frog, TurtleFrog, CanHop o var.                                                                          |
| 3                                               | Falta el punto y coma después de los valores del enum cuando existen otros miembros.                                                                            |
| 4                                               | Una clase que extiende una sealed class debe declararse final, sealed o non-sealed.                                                                             |
| 5                                               | Beetle no implementa correctamente todos los métodos abstractos heredados.                                                                                      |
| 6                                               | Un método abstracto no puede tener cuerpo y una clase implementa una interfaz, no la extiende.                                                                  |
| 7                                               | El método implementado debe ser public para coincidir con la interfaz.                                                                                          |
| 8                                               | Las variables de instancia deben ser private para una encapsulación adecuada.                                                                                   |
| 9                                               | Se aceptan instancias de Cobra, GardenSnake o null.                                                                                                             |
| 10                                              | En X cualquier tipo válido funciona; en Z debe ser ArrayList o subtipo.                                                                                         |
| 11                                              | La salida es 10 porque se usa la variable butter de la clase interna.                                                                                           |
| 12                                              | La encapsulación permite getters y setters y requiere variables privadas.                                                                                       |
| 13                                              | Los case de un enum no deben incluir el nombre del enum.                                                                                                        |
| 14                                              | Las sealed interfaces restringen extensiones e implementaciones; las sealed classes pueden extenderse por clases abstractas.                                    |
| 15                                              | El código no compila porque Spirit es final y no puede extenderse mediante una clase anónima.                                                                   |
| 16                                              | Una clase nested static no puede acceder a miembros de instancia de la clase externa.                                                                           |
| 17                                              | Los métodos de interfaz con cuerpo deben ser default o private; protected no es válido.                                                                         |
| 18                                              | No puede instanciarse la clase interna Diet desde un contexto estático sin una instancia de Deer.                                                               |
| 19                                              | Todos los valores del enum deben implementar el método abstracto.                                                                                               |

|    |                                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------------------|
| 20 | Los métodos sobrescritos se resuelven en tiempo de ejecución; final evita override; los métodos ocultos dependen del tipo de referencia. |
| 21 | Un constructor no compacto de record debe invocar otro constructor como primera instrucción.                                             |
| 22 | Son las sintaxis válidas para crear instancias de clases internas y nested static.                                                       |
| 23 | La sintaxis correcta es Swim.super.perform().                                                                                            |
| 24 | Los métodos static no pueden llamar métodos de instancia directamente.                                                                   |
| 25 | Imprime: x is 24.                                                                                                                        |
| 26 | Los constructores de enum son implícitamente private y falta un punto y coma después de los valores.                                     |
| 27 | Los records generan accessors, equals(), hashCode() y toString().                                                                        |
| 28 | Camel, EatsGrass y Eagle contienen errores de compilación.                                                                               |
| 29 | Existen tres errores de compilación.                                                                                                     |
| 30 | EmperorTamarin y Friendly contienen errores de compilación.                                                                              |